1. First thing we had to was run the YOLO models locally. So we installed the required python package and wrote a script based on the given tutorials.
2. However, this wasn't enough, the accuracy was still quite low and it wasn't able to detect the bicycle.
3. This was because the YOLO library was automatically scaling the images down. Hence, we had to specify the size of the image, post which the models were able to see the correct image rather than the compressed image and hence were able to detect our objects of interest.
4.
5. The code:
6. ```
7. from ultralytics import YOLO
8. import cv2
9.
10. # Load a model
11. model = YOLO("yolo26n.pt") # load an official model
12.
13. source = cv2.imread("images/bicycle.jpg")
14.
15. # Predict with the model
16. results = model.predict(source, imgsz=(2068, 4032)) # predict on an image
17.
18. annotated = None
19. # Access the results
20. for result in results:
21.   xywh = result.boxes.xywh # center-x, center-y, width, height
22.   xywhn = result.boxes.xywhn # normalized
23.   xyxy = result.boxes.xyxy # top-left-x, top-left-y, bottom-right-x, bottom-right-y
24.   xyxyn = result.boxes.xyxyn # normalized
25.   names = [result.names[cls.item()] for cls in result.boxes.cls.int()] # class name of each box
26.   confs = result.boxes.conf # confidence score of each box
27.   bicycle_index = names.index('bicycle')
28.   result.boxes = result.boxes[bicycle_index:bicycle_index+1]
29.   annotated = result.plot()
30.
31. cv2.imwrite('detected.jpg', annotated)
32. '''
33.
34. Next, we had to go find the real world distance between the arch and the Main Building red line. This was a little tricky as we first had to find the locations using Google Street view somehow and estimate approximately which points in the image coincided with the top-down view in google maps.
35. Once we did that, it was straightforward to get the real world distance.

36. Next, we had to draw the lines to get the collinear points. This was fairly straightforward because we just had to trace the fine line between the footpath and the road. By doing this for both left and right sides, we were able to extend the lines and get the vanishing point.
37. Once we got the vanishing point, we had to choose a point in the bounding box which is collinear with the rest of the line on the road. Naturally, this had to be a point on/near the ground and on or on the same level as the bicycle tires.
38. We drew a line from the vanishing point, through the bicycle point, extended all the way to the red line. By using MS Paint, we got the pixel coordinates for all the four points- Vanishing point, point near the third pole, the bicycle and the red line near the MB.
39. Since cross ratio doesn't change regardless of any projective transformation, we equated the cross ratio from pixel world to the real world, using which we were able to get an equation that yielded the distance from the bicycle to the arch.
40. The next section was similar. We photographed our friend in a room. In the photo, the tile lines are clearly visible. The tiles are square and their side length was known to us to be 60cm.

    Using these tile lines, we were able to trace those lines and find our vanishing point. Next, we had to somehow choose a point in the bounding box of our friend from which we were able to draw a line to a collinear point onto another object of interest. This was a little difficult and it shows that it can be tough the find distances for which we don't have any parallel lines or vanishing points directly. Here, we were still able to somehow do it, because we were able to figure out the vanishing for the line from object of interest to our friend.

    Earlier, we tried doing the same work but we didn't place any objects. This definitely made our work harder because we did not have any clear points of reference objects. We had also chosen a wrong non-collinear point on our friend earlier which gave us erroneous results. This led us to question our understanding and concept of vanishing point.

    All in all, it seems at the very least two things are required:

    1. Vanishing point in the direction of the distance that we need to measure.
    2. Reference distance in the same direction, preferably collinear with the distance that we want to measure.